

Agent, Sub-Agent, Skill, or Tool? A Practitioner’s Guide to Extending Agentic AI Systems

Deepak Babu Piskala¹

¹independent researcher

February 25, 2026

Abstract

When designing a new AI capability, practitioners face a fundamental architecture decision: should it be implemented as a tool, a skill, a sub-agent, or a full agent? This seemingly simple question has profound implications for system behavior, debuggability, and maintainability. Yet these terms are often used interchangeably in industry discourse, obscuring the most critical design axis: control over execution. This paper provides clear definitions for tools, skills, sub-agents, and agents based on their control characteristics rather than their perceived intelligence. We introduce the emerging “Skills” abstraction—lightweight, portable modules of procedural knowledge that achieve significant context efficiency gains through progressive disclosure. We present canonical orchestration patterns with practical guidance on when to employ each. Through analysis of prominent frameworks including LangChain, AutoGPT, HuggingGPT, and Voyager, we demonstrate how this taxonomy maps to real implementations. We present detailed case studies from healthcare, gaming, and enterprise domains, each with architectural diagrams illustrating the orchestration decisions. We conclude with a decision framework to help practitioners select the appropriate construct, arguing that autonomy should be earned through necessity rather than assumed by default.

Agent, Sub-Agent, Skill, or Tool?

A Practitioner’s Guide to Extending Agentic AI Systems

Deepak Babu Piskala
Independent Researcher, Seattle, USA

Abstract—When designing a new AI capability, practitioners face a fundamental architecture decision: should it be implemented as a tool, a skill, a sub-agent, or a full agent? This seemingly simple question has profound implications for system behavior, debuggability, and maintainability. Yet these terms are often used interchangeably in industry discourse, obscuring the most critical design axis: *control over execution*. This paper provides clear definitions for tools, skills, sub-agents, and agents based on their control characteristics rather than their perceived intelligence. We introduce the emerging “Skills” abstraction—lightweight, portable modules of procedural knowledge that achieve significant context efficiency gains through progressive disclosure. We present canonical orchestration patterns with practical guidance on when to employ each. Through analysis of prominent frameworks including LangChain, AutoGPT, HuggingGPT, and Voyager, we demonstrate how this taxonomy maps to real implementations. We present detailed case studies from healthcare, gaming, and enterprise domains, each with architectural diagrams illustrating the orchestration decisions. We conclude with a decision framework to help practitioners select the appropriate construct, arguing that autonomy should be earned through necessity rather than assumed by default.

Index Terms—Agentic AI, Multi-Agent Systems, AI Architecture, LLM Orchestration, AI System Design, Tool Use, Skills, Autonomous Agents

I. INTRODUCTION

Over the past two years, “agentic AI” has rapidly become the default architectural pattern for building intelligent systems [9], [13]. Research demonstrations, enterprise copilots, developer tools, and autonomous workflows are now routinely described using the language of agents. Industry reports suggest that 75% of enterprises plan to implement agentic AI by 2027 [14], while academic publications on multi-agent systems have increased exponentially since the introduction of GPT-4 [15].

This shift represents more than a terminological trend—it reflects a genuine evolution in how we build software. Traditional software executes predefined logic. Agentic software, at least in aspiration, *reasons* about its goals and *decides* how to achieve them. The promise is compelling: systems that can handle open-ended tasks, adapt to novel situations, and accomplish complex goals with minimal human oversight.

Yet beneath this surface consensus lies growing ambiguity: systems that behave very differently—operationally, architecturally, and in their failure modes—are described using identical vocabulary. A simple chatbot with function calling is labeled an “agent.” A complex workflow engine with multiple

specialized models is also called an “agent.” An autonomous system that runs continuously without human intervention is—again—an “agent.”

This terminological confusion has practical consequences. Teams adopt “agentic architectures” without understanding what autonomy they are granting or what coordination costs they are incurring. They expect multi-agent systems to be more capable than single-agent systems, only to find performance degradation instead [21]. They build elaborate orchestration layers for problems that could be solved with simple function calls. The result is systems that are harder to debug, more expensive to run, and less reliable than simpler alternatives.

A. The Practitioner’s Dilemma

Consider a software engineer tasked with adding a new capability to an existing AI system. The requirements specify that the system should be able to “research a topic and synthesize findings.” Several architectural choices present themselves:

- Implement it as a **tool**—a function that takes a query and returns search results, leaving synthesis to the main model
- Implement it as a **sub-agent**—a separate reasoning module that searches, filters, and synthesizes, but operates under the control of a parent agent
- Implement it as an **agent**—an autonomous component that decides when research is needed, what to research, and how to integrate findings

Each choice carries different implications. The tool approach is simplest: it provides raw capability without judgment. The calling system must know *when* to search and *what* to do with the results. The sub-agent approach adds reasoning: given a research goal, the sub-agent can decide how to search, what sources to prioritize, and how to synthesize findings. But it remains under the control of its parent—it does not decide whether research is needed in the first place. The agent approach grants full autonomy: the component can observe the conversation, decide that research would be helpful, execute the research, and integrate findings—all without explicit instruction.

This architectural decision is one that most designers stumble upon, often without a clear framework for making the choice. A related question—whether intelligence should live in the agent itself or be distributed across the environment—adds

another dimension to this design space [4]. The wrong decision can lead to systems that are over-engineered, unpredictable, or impossible to debug. Yet the decision is often made based on intuition or industry trends rather than systematic analysis of requirements.

B. The Core Thesis

Key Insight

The key differentiator between tools, sub-agents, and agents is not sophistication or intelligence—it is **who controls the execution flow**. Control, not intelligence, is the real axis along which agentic systems should be understood.

This framing resolves much of the confusion in the field. When practitioners ask “should this be an agent?” they are often asking the wrong question. The right question is: “Who should decide what happens next?” If the answer is “the orchestrator should decide,” then the component should be a tool or sub-agent. If the answer is “the component should decide for itself,” then—and only then—should it be an agent.

The implications extend beyond architecture to operations. Tools can be unit tested. Sub-agents can be integration tested with bounded inputs and outputs. Agents require simulation environments or extensive monitoring because their behavior emerges from an ongoing interaction with their environment. Teams that mistake tools for agents end up building unnecessary infrastructure. Teams that mistake agents for tools end up with systems that fail in unexpected ways because they did not account for autonomous behavior.

When these concepts blur, teams tend to over-engineer solutions. They expect emergent robustness from architectures that cannot produce it, or introduce coordination overhead where none is needed. More importantly, unclear control boundaries make systems harder to reason about, test, and operate [11].

Recent industry analysis supports this concern. Gartner predicted that 75% of firms attempting to build “aspirational agentic architectures” will fail to realize their goals, citing convoluted system designs and unclear governance as primary factors [14]. The solution is not to avoid agentic systems, but to understand precisely what each architectural choice entails. This paper provides that understanding.

C. Contributions

This paper makes the following contributions:

- 1) A precise taxonomy distinguishing tools, sub-agents, and agents based on control characteristics rather than perceived intelligence
- 2) Three canonical orchestration patterns—tool-centric, hierarchical, and decentralized—with clear guidance on applicability
- 3) Analysis of seven prominent frameworks mapped to this taxonomy
- 4) Three detailed case studies with architectural diagrams illustrating real-world orchestration decisions

- 5) A decision framework operationalizing these concepts for practitioners

II. THE CONTROL AXIS

A useful way to think about agentic systems is to ask a single question: *Who decides what happens next?*

This question cuts through the marketing terminology and focuses on what actually matters for system behavior. The answer determines debugging strategies, failure modes, testing approaches, and operational complexity.

A. Defining Control

- If an entity is merely **invoked**, runs a bounded operation, and returns a result, it does not matter how “smart” it appears—it **does not own control**.
- If an entity can **decide** when to act, what to do next, and when to stop, it **does own control**.

Everything else flows from this distinction. The amount of computation, the sophistication of the underlying model, the complexity of the output, and even whether machine learning is involved are all secondary concerns. What matters is whether the component can continue acting without being re-invoked by an external orchestrator.

B. The Control Spectrum

Figure 1 illustrates this spectrum. Components on the left are fully controlled by external entities; components on the right control their own execution. The middle ground—sub-agents—represents an important hybrid: local autonomy within globally controlled boundaries.

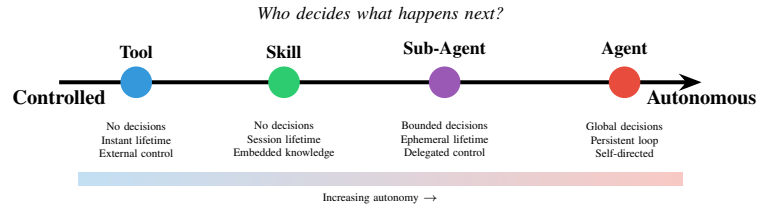


Fig. 1. The control spectrum extended. Skills occupy a unique position: like tools, they provide no decision-making autonomy, but unlike tools, they carry embedded procedural knowledge that persists across a session.

C. Why Control Matters More Than Intelligence

A sophisticated autonomous trading bot that runs continuously but follows fixed rules is less agentic than an LLM researcher that can decide to pursue unexpected lines of inquiry—even if the researcher operates in bounded sessions. The trading bot has a control loop, but it cannot deviate from its rules; the researcher can exercise judgment about *what* to investigate next. This illustrates that agency is not about continuous execution—it is about the capacity to make consequential decisions.

A related distinction separates *workflows* from *agents*. A workflow is a deterministic pipeline: steps execute in a predefined sequence, branches are hardcoded, and the system cannot deviate from its designed path regardless of intermediate

results. An agent, by contrast, can decide what to do next based on observations—the execution path emerges from reasoning rather than being specified in advance. Many systems marketed as “agentic” are actually workflows with LLM steps: sophisticated, yes, but fundamentally predetermined. True agency requires the capacity to choose paths that were not explicitly programmed.

This distinction matters for several reasons. From a **debugging** perspective, tools have bounded failure modes while agents can fail in open-ended ways. For **testing**, tools can be verified with input-output pairs while agents require simulation environments. In **operations**, tools are stateless while agents require monitoring, restarts, and state management. For **governance**, tools execute what they’re told while agents make decisions that need oversight.

III. DEFINITIONS

With the control axis established, we can now precisely define our three constructs. These definitions are grounded in system behavior rather than implementation details or marketing claims.

A. Tool: Capability Without Decision-Making

Definition: Tool

A **tool** is a component that provides capability without agency. It is invoked with structured inputs, performs a bounded operation, and returns an output. It does not reason about *why* it was invoked, *whether* it should retry, or *how* its result should be used. All such decisions live outside the tool.

Examples include:

- Search APIs (Google, Bing, vector databases)
- Database queries (SQL, NoSQL)
- Payment processors (Stripe, PayPal)
- Transcription services (Whisper, AssemblyAI)
- Code execution environments (interpreters, compilers)
- Embedding generators (OpenAI embeddings, Cohere)
- ML inference endpoints (classification, detection)

Even when backed by sophisticated machine learning, these components behave as deterministic—or at least bounded—functions from input to output. A vision model that identifies objects in an image is a tool: it takes an image, returns labels. It does not decide whether to look at another image or what to do with its findings.

From a systems perspective, tools are attractive precisely because they do not own control flow. They are:

- **Predictable:** Given the same inputs, they produce consistent outputs
- **Auditable:** Every invocation can be logged with inputs and outputs
- **Testable:** Unit tests can verify behavior with known inputs
- **Composable:** They can be combined without coordination complexity

In an agentic architecture, tools function like muscles or sensors [17]: powerful, but entirely directed by something else. The analogy is apt—a muscle does not decide when to flex; it executes when commanded. This philosophy echoes principles from Unix system design, where small, composable tools with clear interfaces enable powerful combinations [10].

B. Skill: Embedded Knowledge Without Decision-Making

Definition: Skill

A **skill** is a filesystem-based module containing procedural knowledge—a folder with a Markdown instruction file (SKILL.md) plus optional scripts, references, and assets—that an AI agent can load on demand. Unlike tools, skills carry embedded expertise and workflow guidance. Unlike sub-agents, skills provide no decision-making autonomy; they execute procedures, not judgments.

Skills represent an emerging abstraction introduced by Anthropic in late 2025 [22] and rapidly adopted as an open standard across the industry [23], [25]. They address a critical gap: LLMs have broad knowledge but often lack the specific *procedural* expertise for complex professional tasks. Skills bridge that gap by injecting organization-specific Standard Operating Procedures (SOPs), domain best practices, and tool-specific know-how into the model’s reasoning process—but only when appropriate.

Examples include:

- A **PowerPoint Creator** skill with step-by-step guidelines and templates for company-approved slide design
- A **Legal Contract Review** skill containing compliance checklists and sample clauses
- A **Data Analysis** skill with procedures for loading data, computing metrics, and generating visualizations
- A **PDF Processing** skill with scripts for text extraction, form filling, and document merging

The key innovation is *progressive disclosure*—a three-stage execution model that dramatically improves context efficiency [24]:

- 1) **Discovery:** At startup, the agent loads only each skill’s name and one-line description (50–100 tokens per skill)
- 2) **Activation:** When a user request matches a skill’s scope, the full instructions are loaded into context
- 3) **Execution:** The agent follows the instructions, loading additional references or running embedded scripts only as needed

This lazy loading means skills can greatly expand an AI’s capabilities without permanently consuming context window or computation on every query.

Skills vs. Tools: The distinction is subtle but important. Tools provide *capability* (the ability to search, compute, or execute); skills provide *know-how* (the knowledge of when and how to use capabilities effectively). A skill might instruct the agent to use multiple tools in a specific sequence, with domain-specific judgment at each step. As one analysis noted:

“Tools are atomic operations; skills are composable, high-level capabilities” [26].

Context efficiency: The progressive disclosure model yields dramatic context savings. Traditional tool-heavy architectures require loading all tool definitions upfront—often 15–20% of the context window. Skills achieve the same extensibility with approximately 0.6% overhead until activated, representing a **96% reduction** in context consumption [24].

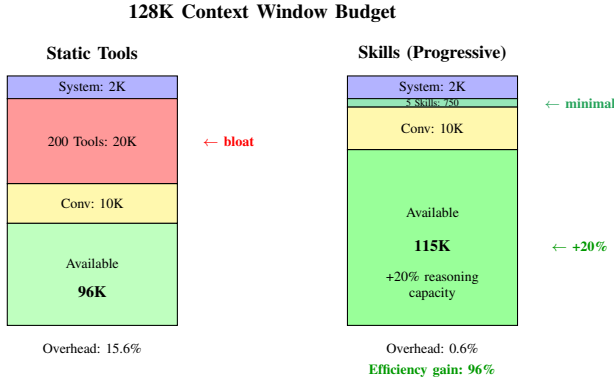


Fig. 2. Context budget comparison: Static tool loading vs. progressive skill disclosure. Skills achieve equivalent extensibility with 96% less context overhead, freeing capacity for reasoning and conversation.

Practitioner’s Tip

Think of skills as *onboarding documents that the AI actually reads*. They capture organizational knowledge in a form that’s both human-auditable and machine-executable—version-controlled Markdown rather than opaque model weights.

C. Sub-Agent: Reasoning Without Autonomy

Definition: Sub-Agent

A **sub-agent** is a reasoning module invoked with a goal and constraints rather than a fixed procedure. It may plan, try multiple approaches, call tools, and critique its own outputs. However, it does not own its lifecycle—it is started by a parent, bounded by a budget, and terminated by the parent. It can think, but it cannot continue thinking unless asked again.

Sub-agents enter the picture when a system needs judgment, iteration, or exploration—but still wants to retain centralized control. They represent *delegated cognition*: the ability to offload complex reasoning to a specialized component without surrendering overall orchestration.

Examples include:

- A **research assistant** that searches the web, filters sources, and synthesizes a brief
- A **debugging unit** that forms hypotheses, runs tests, and proposes fixes

- A **policy interpreter** that evaluates a case against complex regulations
- A **code reviewer** that analyzes changes and suggests improvements
- A **document analyzer** that extracts key information from lengthy texts

These components benefit from reasoning and iteration, but they remain firmly embedded within a single controlling agent’s execution. The parent agent:

- 1) Decides *when* to invoke the sub-agent
- 2) Specifies the *goal* and *constraints*
- 3) Sets *resource limits* (time, tokens, iterations)
- 4) Receives the *result* and decides what to do with it
- 5) Can *terminate* the sub-agent at any point

This pattern is the backbone of most production-ready agentic systems today [11]. It allows systems to decompose complexity, parallelize thinking, and specialize prompts—without introducing the coordination nightmares of distributed control.

Practitioner’s Tip

Think of sub-agents as *consultants*: they bring expertise and can work independently on a problem, but they operate under a contract with clear deliverables and don’t stick around after the engagement ends.

D. Agent: Autonomous Control Loops

Definition: Agent

An **agent** is defined by control closure. It owns a goal, maintains state over time, and decides for itself when to act, what to do next, and when to stop. It is not invoked in the traditional sense; it *runs*. Once started, it continues until its goal is achieved, it is explicitly stopped, or resources are exhausted.

Once an entity can observe its environment, reason about its state, take actions, and continue doing so without being re-invoked, it crosses the threshold into true agency. The key insight from classic AI research remains valid: an agent is characterized by the **observe-think-act loop** that it controls [18], [19].

Examples include:

- **Monitoring agents** that continuously scan metrics and trigger responses
- **Customer resolution agents** that manage support tickets end-to-end
- **Trading agents** that react to market signals autonomously
- **Research agents** like AutoGPT that pursue goals without supervision
- **Embodied agents** like Voyager that explore virtual environments

Agents may use tools and spawn sub-agents as part of their operation, but they do not require an external orchestrator to make progress. The defining feature is not sophistication, but

autonomy over control flow—the agent itself holds the control loop.

The costs of agency: While agents offer maximum flexibility, they introduce significant complexity:

- **Non-determinism:** The same starting conditions may lead to different execution paths
- **Debugging difficulty:** Reproducing failures requires capturing full state trajectories
- **Testing complexity:** Verification requires simulation or extensive logging
- **Operational overhead:** Agents need monitoring, health checks, and restart mechanisms
- **Governance challenges:** Autonomous decisions may require audit trails and kill switches

E. Comprehensive Comparison

Table I summarizes the key differences across eight dimensions.

TABLE I
COMPREHENSIVE COMPARISON OF TOOLS, SKILLS, SUB-AGENTS, AND AGENTS

Dimension	Tool	Skill	Sub-Agent	Agent
Decision	None	None	Bounded	Global
Goal	External	External	Inherited	Own
Control loop	No	No	No	Yes
Lifetime	Instant	Session	Ephemeral	Persistent
Knowledge	None	Embedded	Prompt	Memory
Context cost	High	Low*	Medium	Variable
Debuggability	High	High	Medium	Low

*Skills use progressive disclosure: 50–100 tokens until activated, then full instructions loaded on demand.

IV. ORCHESTRATION PATTERNS

How one arranges agents, sub-agents, and tools determines system behavior. The orchestration pattern—the structure of control flow—is often more important than the capabilities of individual components [11]. Three canonical patterns emerge from our taxonomy.

A. Pattern 1: Tool-Centric Orchestration

In the simplest pattern, a single reasoning loop invokes tools in sequence. There is no delegation of reasoning and no autonomous control. This pattern is often mislabeled as “agentic” but is better understood as *LLM-augmented software* [17].

How it works: The user provides a query, and the agent reasons about which tools to invoke. The agent then calls the selected tool with structured parameters, waits for the tool to execute and return its result, integrates that result into its context, and decides whether the task is complete or whether additional tool calls are needed. This cycle continues until the task is finished, with control never leaving the central agent.

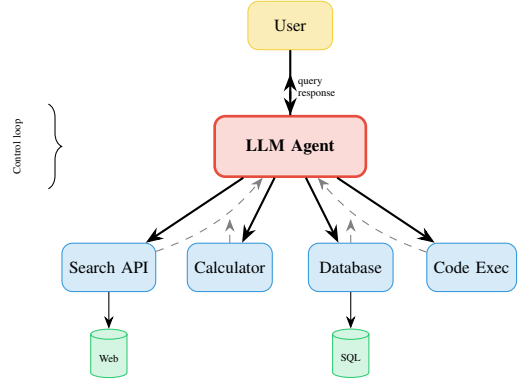


Fig. 3. Tool-centric orchestration: A single agent maintains the control loop, invoking tools as needed. Each tool executes and returns immediately. Control never leaves the central agent.

Strengths:

- Maximum predictability and debuggability
- Clear audit trail of all tool invocations
- Easy to test with mocked tool responses
- Failures are isolated to specific tool calls

Limitations:

- Cannot handle tasks requiring extended reasoning
- Agent context window limits complex multi-step tasks
- No specialization—one prompt must handle everything

This pattern is implemented by most LangChain agents [1], OpenAI’s function calling [13], and the ReAct paradigm [2].

B. Pattern 2: Hierarchical Orchestration

When tasks require specialized reasoning or parallel processing, a hierarchical pattern emerges. A controlling agent delegates bounded reasoning tasks to sub-agents. Control remains centralized, but cognition is modular and specialized.

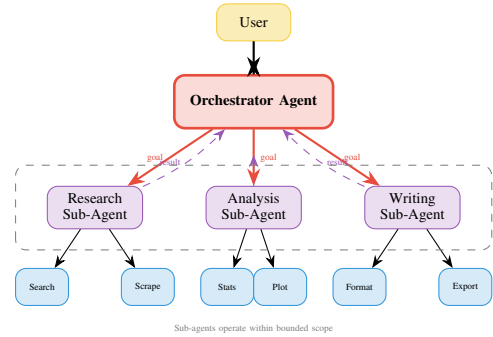


Fig. 4. Hierarchical orchestration: The orchestrator delegates to specialized sub-agents, each with domain-specific tools. Sub-agents reason locally but control returns to the orchestrator.

How it works: The orchestrator receives a complex task and decomposes it into sub-tasks, each with clear goals and constraints. These sub-tasks are assigned to specialized sub-agents, which reason independently, invoke their own tools, and return results when complete. The orchestrator then integrates these results, coordinates any dependencies between

sub-tasks, and either assigns follow-up work or completes the overall task. Throughout this process, control remains centralized in the orchestrator.

Strengths:

- Specialization through domain-specific prompts
- Parallelization of independent sub-tasks
- Bounded complexity per sub-agent
- Centralized coordination and state management

Limitations:

- Added latency from sub-agent invocations
- More complex error handling across components
- Requires careful interface design between layers

This pattern is implemented by HuggingGPT/Jarvis [3], OpenAgents [5], LangChain’s supervisor pattern, and Anthropic’s multi-agent research systems [9].

Practitioner’s Tip

Hierarchical orchestration is where most production-ready agentic systems should live. It provides the benefits of modularity and specialization without the complexity of fully distributed control.

C. Pattern 3: Decentralized Multi-Agent

Only when orchestration itself is decentralized do we arrive at a true multi-agent system. No single entity controls execution; agents coordinate through protocols, negotiation, or shared environments.

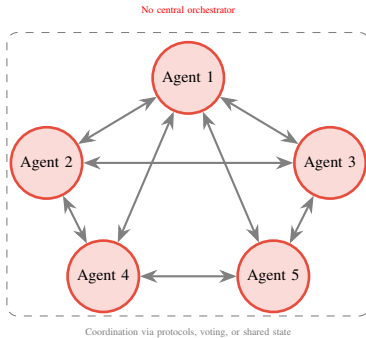


Fig. 5. Decentralized multi-agent: Peer agents coordinate without central control. Each agent owns its control loop. Emergent behavior becomes possible—at the cost of reproducibility.

A simple test helps clarify the boundary: *If the central coordinator disappears, does the system still function?* If not, the system is hierarchical, not truly multi-agent.

When decentralization applies: Decentralized multi-agent architectures are warranted in three scenarios: (1) **distributed ownership**—when agents represent different legal entities with competing interests (supply chain negotiations, energy markets, multi-company logistics); (2) **latency-critical local decisions**—when waiting for a central coordinator would be too slow (fleet routing, smart grid balancing, real-time trading); (3) **fault tolerance**—when no single point of failure is acceptable (distributed manufacturing, peer-to-peer networks).

Coordination without control: In truly decentralized systems, agents produce a final artifact through shared state rather than instruction. Agents observe a common workspace—a task board, codebase, or market state—claim work when ready, and react to others’ outputs. Conflict is resolved through protocols: voting, iteration until convergence, or timeout with defaults. The litmus test remains: if any single agent crashes, do the others continue? In hierarchical systems, they stop. In decentralized systems, they adapt.

Key Insight

Common Misconception: Many “multi-agent” frameworks are actually hierarchical—agents follow a preset sequence where each waits for the previous. True decentralization means agents decide *for themselves* when to act based on observed state, not instruction from an orchestrator.

Key Insight

Multi-agent systems can exhibit emergent behavior, but at significant cost to debuggability and reproducibility. Unless your domain inherently requires decentralization (simulation, negotiation, distributed consensus), prefer hierarchical designs.

V. FRAMEWORKS AND IMPLEMENTATIONS

Several prominent frameworks implement these patterns. Understanding how they map to our taxonomy helps practitioners make informed choices. Table II provides a summary.

VI. CASE STUDIES

To illustrate how organizations have applied this taxonomy in practice, we examine three real-world systems that made deliberate orchestration choices. Each case study includes an architectural diagram showing the complete system design.

A. Case Study 1: Healthcare—Microsoft’s Cancer Care Orchestrator

Domain: Oncology treatment planning

Pattern chosen: Hierarchical orchestration

Key decision: Sub-agents over autonomous agents

Rationale: Accountability and traceability were non-negotiable

Microsoft developed a Healthcare Agent Orchestrator [12] to assist clinicians in cancer treatment planning—specifically for tumor board reviews where specialists must synthesize patient data across imaging, genomics, clinical notes, and lab results.

The Challenge: Cancer treatment planning requires integrating multiple data modalities: radiology images, genomic sequencing results, pathology reports, clinical notes, and lab values. No single AI model handles all these data types

TABLE II
REPRESENTATIVE SYSTEMS MAPPED TO ORCHESTRATION PATTERNS

System / Paradigm	Pattern	Key Characteristic
ReAct [2]	Tool-centric	Interleaved reasoning + action; single control loop
HuggingGPT / Jarvis [3]	Hierarchical	LLM orchestrates specialized ML models as sub-agents
OpenAgents [5]	Hierarchical	Coordinator agent delegates to domain-specific agents
AutoGPT [6]	Autonomous	Self-directed goal pursuit without human supervision
Voyager [7]	Autonomous	Lifelong learning with persistent skill library
MetaGPT [8]	Multi-Agent	Role-based collaboration across software lifecycle
CAMEL [20]	Multi-Agent	Role-playing conversations between peer agents

Note: Implementation frameworks like LangChain, CrewAI, and OpenAI's Assistants API can realize any of these patterns; they are architectural building blocks, not pattern exemplars.

effectively. Yet the final recommendation must be traceable—clinicians need to understand *why* a particular treatment was suggested.

Architectural Decision: Rather than building a single monolithic agent or multiple autonomous agents, Microsoft chose *hierarchical orchestration with specialized sub-agents*.

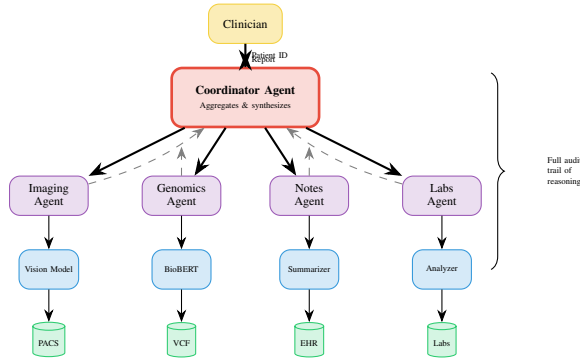


Fig. 6. Microsoft Healthcare Agent Orchestrator architecture. The Coordinator Agent delegates to specialized sub-agents for each data modality, then synthesizes findings. Every reasoning step is logged for clinical accountability.

Why Hierarchical? The healthcare domain demanded both specialization and *centralized accountability*:

- **Tool-centric would fail** because interpreting radiology images requires multi-step reasoning (identify tumor, assess margins, compare to prior scans)—beyond what a simple tool call can provide
- **Autonomous agents would fail** because clinical recommendations must be traceable; a decentralized system would make it impossible to explain how a recommendation was derived
- **Hierarchical succeeded** because it allowed domain-specific reasoning (each sub-agent specializes) while keeping a single coordinator responsible for the final output

Outcome: The system reduced clinician review time from hours to minutes. More importantly, every recommendation includes a reasoning trace showing which sub-agent contributed what findings—essential for clinical oversight and liability.

B. Case Study 2: Gaming—Voyager’s Autonomous Exploration

Domain: Open-world game playing (Minecraft)

Pattern chosen: Autonomous agent with tool use

Key decision: Full autonomy over hierarchical delegation

Rationale: Open-ended environment with no predefined subtasks

Voyager [7] is an autonomous agent that learns to play Minecraft without human intervention. Unlike most game-playing AI, Voyager has no predefined curriculum—it must decide what to explore, what skills to acquire, and how to adapt to an open-ended environment.

The Challenge: Minecraft is an open-world environment with effectively infinite possibilities. There are no predefined levels, objectives, or win conditions. The agent must:

- Discover what goals are even possible
- Learn skills progressively (can’t craft diamond tools without first learning to mine)
- Adapt to novel situations (different biomes, enemies, resources)
- Retain and reuse learned skills

Architectural Decision: The researchers chose a *single autonomous agent* with sophisticated internal mechanisms rather than multiple coordinating agents.

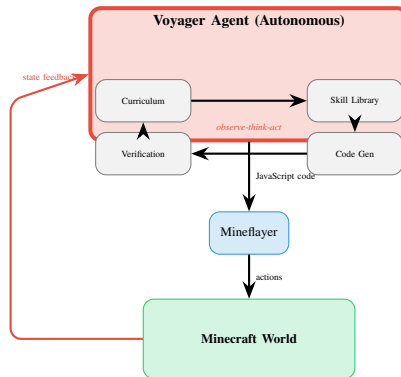


Fig. 7. Voyager architecture. A single autonomous agent contains internal components for curriculum generation, code generation, verification, and skill storage. The agent runs continuously in an observe-think-act loop, interacting with Minecraft through code execution.

Why Autonomous? The open-ended nature of Minecraft made other patterns unsuitable:

- **Tool-centric would fail** because there’s no fixed set of actions—the agent needs to *create* new tools (skills) as it discovers new possibilities
- **Hierarchical would fail** because you can’t predefine sub-agents for an open-ended world; the agent needed to discover what specializations were even useful
- **Autonomous succeeded** because: (1) the environment is dynamic and unpredictable, (2) failures are cheap (dying in-game has no real cost), and (3) the research goal explicitly required self-directed exploration

Internal Mechanisms: What makes Voyager work is not raw autonomy but well-designed internal structure:

- **Automatic curriculum:** Generates progressively harder goals based on current capabilities
- **Skill library:** Stores successful code solutions for reuse (effectively, learned tools)
- **Self-verification:** Checks whether goals were achieved before moving on
- **Code generation:** Creates JavaScript to control the Minecraft character

Outcome: Voyager acquired $3.3\times$ more unique items than baselines and demonstrated genuine lifelong learning. It validated that autonomy, when the domain warrants it, can achieve results impossible with tighter control.

C. Case Study 3: Enterprise—Customer Support with Tool-Centric Design

Domain: Enterprise customer service
Pattern chosen: Tool-centric orchestration
Key decision: Tools over sub-agents
Rationale: Predictability and compliance over capability

Many enterprise customer support systems—including those built on LangChain and similar frameworks—deliberately choose the simplest pattern: a single conversational agent that calls tools.

The Challenge: Enterprise customer service has stringent requirements:

- Every interaction must be logged for compliance
- Agents cannot make unauthorized commitments
- Failures must be traceable to specific components
- Response time SLAs must be predictable
- Integration with existing CRM, ticketing, and knowledge systems

Architectural Decision: A *tool-centric pattern* with carefully designed, auditable tools.

Why Tool-Centric? Enterprise environments prioritize *predictability over capability*:

- **Sub-agents would introduce uncertainty:** What if a research sub-agent returned inconsistent information? What if a planning sub-agent made unauthorized commitments to customers?
- **Autonomous agents would be dangerous:** An agent that can “reason itself” into novel behaviors is a compliance nightmare

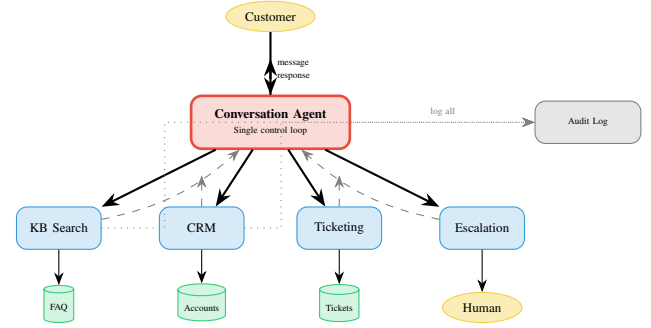


Fig. 8. Enterprise customer support architecture. A single conversation agent calls well-defined tools for knowledge retrieval, CRM access, ticketing, and escalation. Every tool call is logged for compliance and debugging.

- **Tools are auditable:** Every action is an explicit tool call with logged inputs and outputs

Design Principles:

- 1) **Tools are verbs:** Each tool represents a single, well-defined action (search, lookup, create, escalate)
- 2) **No hidden reasoning:** The agent’s decision to call a tool is explicit and logged
- 3) **Fail-safe defaults:** When uncertain, escalate to humans rather than attempt novel reasoning
- 4) **Bounded scope:** Tools cannot perform actions outside their defined capabilities

Outcome: Tool-centric support agents handle 60-80% of routine queries autonomously while maintaining compliance guarantees. When they escalate, human agents receive full context of what was tried.

D. Case Study 4: Anti-Pattern—When Multi-Agent Made Things Worse

Domain: Sequential reasoning benchmarks
Pattern attempted: Multi-agent coordination
Key finding: Performance degraded 39–70%
Lesson: MAS is not universally superior

Kim et al. [21] conducted a systematic study across 180 agent configurations, comparing single-agent and multi-agent approaches on tasks including finance, web navigation, planning, and general workbench tasks.

The Expectation: Adding more agents would improve performance through specialization, parallel processing, and distributed reasoning. The intuition was that complex tasks benefit from “divide and conquer.”

The Reality: For **sequential reasoning tasks**, every multi-agent variant tested *degraded* performance by 39–70% compared to single-agent baselines. The study identified three interconnected failure modes that help explain this counter-intuitive result.

The first is the *tool-coordination tradeoff*. Under fixed computational budgets, tool-heavy tasks suffered disproportionately from multi-agent overhead. When agents must coordinate before and after each tool use, the communication

cost compounds with every action. For tasks requiring many sequential tool calls—common in enterprise workflows—this overhead quickly exceeded any benefit from parallelization.

The second failure mode is *capability saturation*. Once single-agent baselines exceeded approximately 45% accuracy on a task, adding more agents yielded diminishing or even negative returns. This threshold represents a tipping point: below it, coordination can help agents avoid errors; above it, the communication overhead outweighs any collective intelligence gains. Most production-ready LLMs already exceed this threshold on straightforward tasks.

The third and perhaps most damaging failure mode is *error amplification*. When agents operate independently, errors in one agent’s reasoning propagate to others, compounding through the system. The study measured error amplification of $17.2\times$ for independent agent topologies. Even centralized coordination—where a supervisor agent attempts to catch and correct errors—only reduced this to $4.4\times$. The implication is sobering: multi-agent systems can make your most capable models appear less capable.

That said, multi-agent coordination is not universally harmful. The study found that decentralized coordination genuinely excelled on web navigation tasks, improving performance by 9.2% compared to just 0.2% for centralized approaches. The key difference? Web navigation benefits from parallel exploration of different paths—a genuinely parallelizable task where agents can independently pursue different strategies without needing to coordinate at every step. Similarly, centralized coordination improved performance by 80.8% on explicitly parallelizable tasks where subtasks had minimal dependencies.

The lesson for practitioners is nuanced but clear: multi-agent systems are not inherently superior, and the assumption that “more agents means more capability” can lead to significant performance degradation. MAS architectures excel when tasks are genuinely parallelizable and subtask independence is high. For sequential reasoning—which characterizes the majority of enterprise workflows, from customer support to document processing—single-agent designs with appropriate tools consistently outperform their multi-agent counterparts.

Practitioner’s Tip

Before adding agents, ask: Is this task genuinely parallelizable? If tasks must execute sequentially, MAS coordination overhead will likely hurt more than help.

E. Lessons from the Case Studies

These cases reveal common patterns in orchestration decisions:

The pattern emerges clearly: *match autonomy to domain requirements*:

- High-stakes, auditable domains → Hierarchical or tool-centric
- Open-ended, failure-tolerant domains → Autonomous
- Compliance-critical domains → Tool-centric

TABLE III
CASE STUDY SUMMARY: MATCHING PATTERN TO DOMAIN

Case	Pattern	Key Driver	Would Fail
Healthcare	Hierarchical	Accountability	Autonomous (no trace)
Voyager	Autonomous	Open-ended	Hierarchical (rigid)
Enterprise	Tool-centric	Compliance	Sub-agents (risk)
Kim et al.	Single-agent	Sequential tasks	MAS (39–70% worse)

VII. THE MULTI-AGENT TEMPTATION

Before presenting the decision framework, we must address a common pitfall: the assumption that more agents equals more capability.

A. The Sophistication Fallacy

There is a pervasive belief that multi-agent systems are inherently more sophisticated than single-agent designs. This conflates *architectural complexity* with *capability*. In reality, multi-agent coordination introduces overhead that often exceeds its benefits.

Recent empirical research confirms this concern. Kim et al. [21] conducted a systematic evaluation across 180 agent configurations and found that **for sequential reasoning tasks, every multi-agent variant degraded performance by 39–70%** compared to single-agent baselines. Their study identified “capability saturation”—the point at which coordination yields diminishing or even negative returns, typically when single-agent performance exceeds 45%.

Key Insight

More agents $\neq$ more intelligent. A well-designed single agent with appropriate tools often outperforms a poorly-coordinated multi-agent system in both performance and reliability. Empirical evidence shows multi-agent coordination degrades performance by 39–70% on sequential tasks [21].

B. Hidden Costs of Multi-Agent Systems

Table IV summarizes the costs that multi-agent architectures introduce beyond the obvious computational overhead.

C. Signs You’re Over-Engineering

Several indicators suggest that a simpler architecture would suffice. The most telling sign is when agents spend more time coordinating than actually working on the task. If the overhead of agent-to-agent communication exceeds the computation devoted to the actual problem, the architecture is likely over-engineered.

Another red flag emerges when developers cannot articulate why a particular capability needs its own agent rather than

TABLE IV
HIDDEN COSTS OF MULTI-AGENT ARCHITECTURES

Cost Type	Description
Latency	Each agent handoff adds round-trip time
Token waste	Coordination messages consume context
Debugging	Failures span multiple components
State sync	Agents may have inconsistent views
Testing	Combinatorial explosion of interactions

being implemented as a tool. If the justification reduces to “it seemed more sophisticated” or “agents are the modern approach,” this is a design smell rather than a design rationale.

Perhaps the most diagnostic test is attempting to replace an “agent” with a simple function call. If doing so does not change the observable behavior of the system—if the same inputs still produce the same outputs—then what was labeled an “agent” was never truly autonomous. It was an over-engineered tool masquerading as something more sophisticated.

Finally, examine what agents are actually communicating. If most inter-agent messages consist of passing data structures rather than genuine negotiation, reasoning, or coordination, the system is likely using agents where simple function composition would suffice.

Practitioner’s Tip

The MAS Test: If you can replace an “agent” with a function call and the system behavior remains unchanged, it was never really an agent—it was an over-engineered tool.

VIII. DECISION FRAMEWORK

When should you use a tool, sub-agent, or agent? Figure 9 provides a practical decision tree, and the subsections below offer detailed guidelines. Beyond the architectural choice, practitioners must also consider which underlying models to employ—a decision that involves balancing performance, cost, latency, and even ethical considerations [16].

Key Insight

The Golden Rule: Prefer the least autonomous approach that satisfactorily solves the problem. Autonomy should be earned through demonstrated necessity, not assumed as default.

IX. CONCLUSION

As AI systems become more agentic, practitioners need a clear taxonomy grounded in system behavior rather than marketing terminology. The framework presented here—distinguishing tools, skills, sub-agents, and agents by

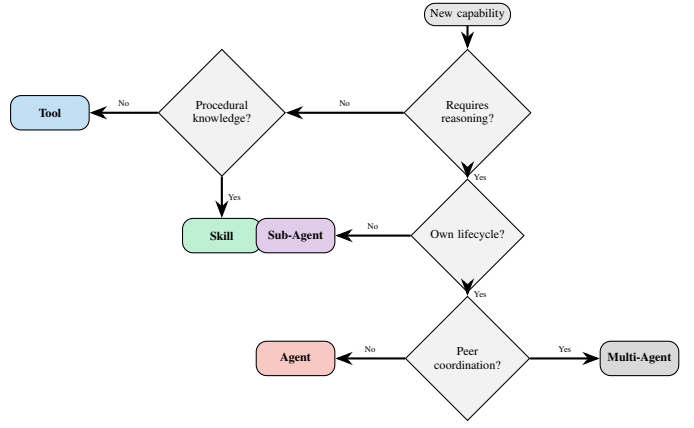


Fig. 9. Decision flowchart for selecting the appropriate construct. Skills are the right choice when procedural knowledge is needed but no reasoning or decision-making autonomy. Start with the simplest option that meets requirements.

their control characteristics—provides actionable guidance for architectural decisions.

Key insights:

- 1) **Control, not intelligence, is the real axis.** The defining question is “Who decides what happens next?”
- 2) **Tools provide capability, skills provide know-how, sub-agents provide bounded reasoning, agents provide autonomy.** Each has its place; using the wrong abstraction creates unnecessary complexity.
- 3) **Skills represent a paradigm shift** in context efficiency—achieving 96% reduction in context overhead through progressive disclosure while capturing organizational knowledge in auditable, version-controlled form.
- 4) **Hierarchical orchestration is the sweet spot** for most production systems—providing modularity without distributed control.
- 5) **Autonomy should be earned, not assumed.** Multi-agent architectures should be reserved for problems that inherently require decentralization.
- 6) **Match pattern to domain:** Healthcare chose hierarchy for accountability; Voyager chose autonomy for open-ended exploration; enterprise chose tools for compliance.

Understanding this distinction is the difference between an elegant agentic architecture and an accidental distributed system. Clear taxonomy leads to clearer ownership, simpler debugging, and more maintainable systems.

REFERENCES

- [1] LangChain Documentation, “Agents and Tools,” 2024. [Online]. Available: <https://docs.langchain.com>
- [2] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” arXiv:2210.03629, 2023.
- [3] Y. Shen et al., “HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in HuggingFace,” arXiv:2303.17580, 2023.
- [4] DB Piskala, “Where Should Intelligence Live? Agent-Centric vs. Environment-Centric Design in Agentic Systems,” TechRxiv, 2025. DOI: 10.36227/techrxiv.176823084.40754167
- [5] xLang AI, “OpenAgents: An Open Platform for Language Agents in the Wild,” COLM 2024, arXiv:2310.10634.

- [6] Significant Gravitas, “Auto-GPT,” GitHub Repository, 2023.
- [7] G. Wang et al., “Voyager: An Open-Ended Embodied Agent with Large Language Models,” arXiv:2305.16291, 2023.
- [8] S. Hong et al., “MetaGPT: Meta Programming for Multi-Agent Collaborative Framework,” arXiv:2308.00352, 2023.
- [9] Anthropic, “Building Effective Agents,” Technical Report, 2024.
- [10] DB Piskala, “From Everything-is-a-File to Files-Are-All-You-Need: How Unix Philosophy Informs the Design of Agentic AI Systems,” arXiv:2601.11672, 2026.
- [11] S. Runkle, “Choosing the Right Multi-Agent Architecture,” LangChain Blog, 2024.
- [12] Microsoft, “Developing next-generation cancer care management with multi-agent orchestration,” Microsoft Industry Blog, May 2025.
- [13] OpenAI, “Function Calling and Agents,” OpenAI Documentation, 2024.
- [14] Gartner, “Predicts 2025: AI Agents Will Transform Enterprise Operations,” Gartner Research, 2024.
- [15] Z. Xi et al., “The Rise and Potential of Large Language Model Based Agents: A Survey,” arXiv:2309.07864, 2023.
- [16] DB Piskala, V. Raajaa, S. Mishra, and B. Bozza, “Dynamic LLM Routing and Selection based on User Preferences: Balancing Performance, Cost, and Ethics,” arXiv:2502.16696, 2025.
- [17] G. Mialon et al., “Augmented Language Models: a Survey,” arXiv:2302.07842, 2023.
- [18] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020.
- [19] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed., Wiley, 2009.
- [20] G. Li et al., “CAMEL: Communicative Agents for Mind Exploration of Large Language Model Society,” NeurIPS 2023.
- [21] Y. Kim et al., “Towards a Science of Scaling Agent Systems,” arXiv:2512.08296, 2025.
- [22] Anthropic, “Introducing Agent Skills,” Claude Blog, October 2025. [Online]. Available: <https://claude.com/blog/skills>
- [23] Anthropic, “Agent Skills Specification v1.0,” 2025. [Online]. Available: <https://agentskills.io/specification>
- [24] S. Willison, “Claude Skills are awesome, maybe a bigger deal than MCP,” simonwillison.net, October 2025.
- [25] M. Nuñez, “Anthropic launches enterprise Agent Skills and opens the standard,” VentureBeat, December 2025.
- [26] A. Laurent, “Claude Skills vs. MCP: A Technical Comparison for AI Workflows,” IntuitionLabs, January 2026.